WEEK 4 · SIMPLE GUIDE

C++ Concurrency, Step by Step

Only the topics you need, explained in plain language with short examples.

What you will learn: threads · race conditions · critical sections · mutexes · `lock_guard` · `unique_lock` · deadlocks · condition variables · producer–consumer · futures and promises · atomics · thread-safe classes.

What you will build: a small thread pool.

Every example is short (most are under 25 lines), complete, and tested. Type them in, run them, change them.

Contents

1. What Is a Thread? 4

How to compile the examples . . . 4

2. Starting Threads 5

2.1 Your first thread . . . 5

2.2 Several threads, with arguments . . . 5

3. The Big Problem: Race Conditions 7

3.1 A broken program . . . 7

3.2 Why it breaks . . . 7

3.3 Critical sections . . . 7

4. Mutexes: Taking Turns 8

4.1 The idea . . . 8

4.2 Fixing the counter with lock_guard . . . 8

4.3 unique_lock: a more flexible lock . . . 8

4.4 Two simple habits . . . 9

5. Deadlocks 10

5.1 What is a deadlock? . . . 10

5.2 The fix . . . 10

6. Condition Variables: Waiting for a Signal 12

6.1 The problem . . . 12

6.2 Example . . . 12

7. Producer–Consumer 14

8. Futures and Promises: Getting a Result Back 16

8.1 std::async — the easy way . . . 16

8.2 std::promise — sending the value yourself . . . 16

9. Atomics 18

10. Thread-Safe Classes 19

11. Project: A Simple Thread Pool 20

11.1 What is a thread pool? . . . 20

11.2 Build it in 4 steps . . . 20

11.3 The complete thread pool . . . 21

11.4 The important details . . . 22

11.5 Make it your own . . . 22

12. Summary 23

12.1 Which tool do I use? . . . 23

12.2 The most common mistakes 23
12.3 Finding race conditions automatically 23
13. Where to Learn More **24**

1. What Is a Thread?

Normally your program does one thing at a time, line by line, from the top of `main` to the bottom. A **thread** is a separate line of work that runs **at the same time** as the rest of your program.

> **KEY IDEA**
>
> **Picture a kitchen.** Your program is the kitchen. Each thread is a cook. Several cooks can work at the same time, which gets dinner ready faster. But they **share** the same fridge, the same counter and the same pots. If two cooks grab the same pot at the same moment, things go wrong. Almost everything this week is about how cooks share the kitchen without getting in each other's way.

Why use threads?

- **Speed:** split a big job into pieces and work on them at the same time on different CPU cores.
- **Not waiting:** while one thread waits for something slow (a file, the network), another can keep working.

What do threads share? All threads in a program share **global variables and heap memory** (anything made with `new`, the contents of a `vector`, and so on). Each thread has its **own local variables**. Shared things need care; local things are safe.

How to compile the examples

Threads need the `-pthread` flag on Linux and macOS:

```
g++ -std=c++17 -Wall -pthread example.cpp -o example
./example
```

> **TRY IT**
>
> Run every example **a few times**. With threads, the output order can change from run to run. That is normal, and seeing it with your own eyes is the best way to understand it.

2. Starting Threads

2.1 Your first thread

01_first_thread.cpp

```cpp
#include <iostream>
#include <thread>
void say_hello() {
    std::cout << "Hello from the new thread!\n";
}
int main() {
    std::thread t(say_hello);   // start a new thread running say_hello
    std::cout << "Hello from main!\n";
    t.join();                   // wait here until the thread is finished
    std::cout << "Thread finished.\n";
}
```

What happens, line by line:

Line	What it does
`#include <thread>`	Gives you `std::thread`.
`std::thread t(say_hello);`	Creates a new thread and **immediately** starts running `say_hello` on it. `main` keeps going at the same time.
`t.join();`	`main` **waits** here until the thread has finished.

Possible output (the first two lines may come in either order):

```
Hello from main!
Hello from the new thread!
Thread finished.
```

PITFALL

Always call `join()` before the thread object goes away. If you forget, your program crashes (it calls `std::terminate`). Rule for this week: every thread you start, you join.

2.2 Several threads, with arguments

02_many_threads.cpp

```cpp
#include <iostream>
#include <thread>
#include <vector>
void work(int id) {
    std::cout << "Worker " + std::to_string(id) + " is working\n";
}
int main() {
    std::vector<std::thread> threads;
    for (int i = 1; i <= 4; i++) {
        threads.push_back(std::thread(work, i));   // pass i as argument
    }
    for (auto& t : threads) {
        t.join();                                  // wait for each one
    }
    std::cout << "All workers done\n";
}
```

- `std::thread(work, i)` starts `work(i)` on a new thread. Extra arguments go after the function name.

- We keep the threads in a `vector`, then join them all in a second loop. Starting them all first and joining later lets them run **at the same time**.
- Each line is built as one string before printing, so lines from different threads don't get mixed in the middle.

You can also give a thread a **lambda** (a small unnamed function), which is very common:

```
std::thread t([] {
    std::cout << "Hi from a lambda\n";
});
t.join();
```

TRY IT

Change the loop to start 8 workers. Run it 5 times. Is the order always the same?

3. The Big Problem: Race Conditions

3.1 A broken program

Two threads each add 1 to a shared counter, 100,000 times. The answer should be 200,000.

03_race.cpp

```cpp
#include <iostream>
#include <thread>
int counter = 0;            // shared by both threads
void add_many() {
    for (int i = 0; i < 100000; i++) {
        counter++;          // NOT safe: two threads change it at once
    }
}
int main() {
    std::thread a(add_many);
    std::thread b(add_many);
    a.join();
    b.join();
    std::cout << "counter = " << counter << "\n";  // expected 200000
}
```

On a computer with several cores you will usually get a **smaller** number, and a **different** one each run, for example `counter = 131072`, then `counter = 158233`. (On a single-core machine it may look correct by luck. That is what makes these bugs dangerous.)

3.2 Why it breaks

`counter++` looks like one step, but the computer does it in **three** steps: (1) read the value, (2) add 1, (3) write the value back. Two threads can mix up these steps:

Step	Thread A	Thread B	counter
1	reads 5		5
2		reads 5	5
3	adds 1 → 6		5
4		adds 1 → 6	5
5	writes 6		6
6		writes 6	**6** (should be 7!)

Both threads added 1, but the counter only went up by 1. One update was **lost**.

> **KEY IDEA**
>
> A **race condition** is a bug where the result depends on the timing of threads — who happens to go first. It happens when threads use **shared data** and at least one of them **changes** it.

3.3 Critical sections

A **critical section** is a piece of code that uses shared data and must not be run by two threads at once. In our example, `counter++` is the critical section. The fix is to make sure only **one thread at a time** can be inside it. That is exactly what a **mutex** does.

4. Mutexes: Taking Turns

4.1 The idea

> **KEY IDEA**
>
> **A mutex is like the single key to a bathroom.** Whoever has the key goes in; everyone else waits at the door. When they come out, they hand back the key and the next person goes in. Only one person inside at a time.

In C++: `std::mutex` is the key. **Locking** it = taking the key (wait if someone else has it). **Unlocking** it = giving it back.

4.2 Fixing the counter with lock_guard

04_mutex.cpp

```cpp
#include <iostream>
#include <mutex>
#include <thread>
int counter = 0;
std::mutex counter_mutex;   // protects counter
void add_many() {
    for (int i = 0; i < 100000; i++) {
        std::lock_guard<std::mutex> lock(counter_mutex);  // lock
        counter++;                                        // safe now
    }                                                     // unlock here
}
int main() {
    std::thread a(add_many);
    std::thread b(add_many);
    a.join();
    b.join();
    std::cout << "counter = " << counter << "\n";  // always 200000
}
```

- `std::mutex counter_mutex;` — one mutex that protects `counter`.
- `std::lock_guard<std::mutex> lock(counter_mutex);` — **locks** the mutex when created.
- At the closing `}` the `lock` variable is destroyed, which **unlocks** the mutex automatically.
- Now only one thread at a time can run `counter++`, so the answer is always 200,000.

> **RULE OF THUMB**
>
> Always use `lock_guard` (or `unique_lock`) instead of calling `lock()` and `unlock()` yourself. If you forget `unlock()`, or an error happens before it, the mutex stays locked forever and the program freezes. `lock_guard` unlocks for you, every time. (This is RAII from Week 2.)

4.3 unique_lock: a more flexible lock

`std::unique_lock` does the same job as `lock_guard`, but you can **unlock it early** and lock it again. You need it for condition variables (Section 6).

```
std::unique_lock<std::mutex> lock(m);   // locked
shared_list.push_back(x);
lock.unlock();                          // unlocked early
do_slow_work();                         // others can use the list now
lock.lock();                            // locked again
```

Use	When
`std::lock_guard`	Most of the time. Lock for the whole block, unlock at the end.
`std::unique_lock`	You need to unlock early, or you are using a condition variable.

4.4 Two simple habits

- **Lock as little as possible.** Only put the code that touches shared data inside the lock. Slow work (printing, reading files, big calculations) should happen outside, or other threads wait for nothing.
- **One mutex for one piece of shared data.** Keep them together, ideally in the same class (Section 9).

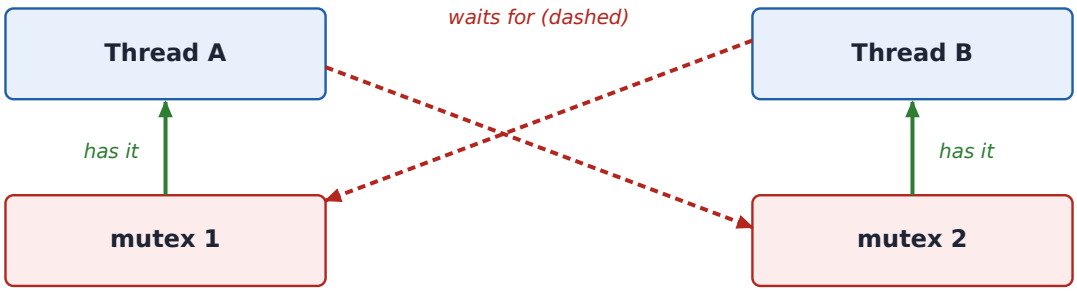
5. Deadlocks

5.1 What is a deadlock?

A **deadlock** happens when threads wait for each other forever. The program doesn't crash — it just **freezes**.

KEY IDEA

Two people, one table, two forks. You need both forks to eat. Person A picks up fork 1 and waits for fork 2. Person B picks up fork 2 and waits for fork 1. Each is holding what the other needs. Nobody ever eats.



Each thread holds one mutex and waits for the other one. Forever.

In code, this is the dangerous pattern — the two functions lock in **opposite order**:

```
void person_a() {
    std::lock_guard<std::mutex> l1(fork1);   // A takes fork 1
    std::lock_guard<std::mutex> l2(fork2);   // ...waits for fork 2
}
void person_b() {
    std::lock_guard<std::mutex> l1(fork2);   // B takes fork 2
    std::lock_guard<std::mutex> l2(fork1);   // ...waits for fork 1
}
```

5.2 The fix

05_deadlock_fix.cpp

```
#include <iostream>
#include <mutex>
#include <thread>
std::mutex fork1, fork2;
void person_a() {
    std::scoped_lock both(fork1, fork2);   // takes BOTH safely
    std::cout << "A is eating\n";
}
void person_b() {
    std::scoped_lock both(fork2, fork1);   // order doesn't matter here
    std::cout << "B is eating\n";
}
int main() {
    std::thread a(person_a);
    std::thread b(person_b);
    a.join();
    b.join();
}
```

`std::scoped_lock` (C++17) locks **several mutexes at once** in a safe way, so it can never get stuck halfway. The order you list them in doesn't matter.

> **RULE OF THUMB**
>
> Three rules that prevent almost all deadlocks: **(1)** if you need two mutexes, lock them together with `std::scoped_lock`. **(2)** If you can't, always lock them in the **same order** everywhere. **(3)** Don't lock a mutex you already hold — a plain `std::mutex` locked twice by the same thread freezes.

6. Condition Variables: Waiting for a Signal

6.1 The problem

Sometimes a thread must **wait until something happens**: data is ready, a queue has items, it's time to stop. Checking again and again in a loop wastes the CPU. Instead, we want the thread to **sleep** and be **woken up** when something changes.

> **KEY IDEA**
>
> **A restaurant buzzer.** You order, sit down and relax (sleep). When your food is ready, the buzzer goes off (notify) and you go and check the counter (check the condition).

6.2 Example

06_condition_variable.cpp

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <thread>
std::mutex m;
std::condition_variable cv;
bool ready = false;            // the thing we are waiting for
void waiter() {
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock, [] { return ready; });   // sleep until ready is true
    std::cout << "Waiter: got the signal, continuing!\n";
}
void signaler() {
    std::cout << "Signaler: preparing...\n";
    {
        std::lock_guard<std::mutex> lock(m);
        ready = true;                      // change the state
    }
    cv.notify_one();                       // wake up the waiter
}
int main() {
    std::thread w(waiter);
    std::thread s(signaler);
    w.join();
    s.join();
}
```

You always need **three things together**:

1. A **mutex** (`m`).
2. A **shared variable** that describes the situation (`ready`).
3. A **condition variable** (`cv`) to sleep on and to wake others up.

What happens:

- **Waiter:** locks `m`, then `cv.wait(lock, [] { return ready; })`. If `ready` is false, it **unlocks the mutex and goes to sleep**. When woken, it locks again and re-checks `ready`. It only continues when `ready` is true.
- **Signaler:** locks `m`, sets `ready = true`, unlocks, then calls `cv.notify_one()` to wake one waiting thread.

PITFALL

Always give `wait` a condition (the `[] { return ready; }` part). A thread can sometimes wake up for no reason (“spurious wakeup”), and the condition check makes it go back to sleep. Also, always change `ready` **while holding the mutex**, otherwise the signal can be missed.

`notify_one()` wakes one waiting thread. `notify_all()` wakes all of them — use it when everyone needs to know, for example “we are shutting down”.

7. Producer–Consumer

This is the most useful pattern of the week. One thread **produces** items and puts them in a shared queue. Another thread **consumes** them. The queue sits in the middle.

KEY IDEA

A conveyor belt. One worker puts boxes on the belt; another takes them off and packs them. They don't have to work at the same speed. If the belt is empty, the packer waits.



shared queue (protected by a mutex)

The producer pushes into the queue; the consumer pops from it.

07_producer_consumer.cpp

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
std::queue<int> boxes;          // the shared "conveyor belt"
std::mutex m;
std::condition_variable cv;
bool finished = false;          // producer sets this when done
void producer() {
    for (int i = 1; i <= 5; i++) {
        {
            std::lock_guard<std::mutex> lock(m);
            boxes.push(i);
            std::cout << "Produced " << i << "\n";
        }
        cv.notify_one();        // tell the consumer there is a box
    }
    {
        std::lock_guard<std::mutex> lock(m);
        finished = true;
    }
    cv.notify_one();
}
void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock, [] { return !boxes.empty() || finished; });
        if (boxes.empty()) break;           // finished and nothing left
        int box = boxes.front();
        boxes.pop();
        lock.unlock();                      // don't hold lock while working
        std::cout << "  Consumed " << box << "\n";
    }
}
int main() {
    std::thread p(producer);
    std::thread c(consumer);
    p.join();
    c.join();
}
```

Everything here uses what you already know:

- The **queue** `boxes` is shared, so every access is inside a lock.

- **Producer:** lock, push a box, unlock, `notify_one()` to wake the consumer. At the end it sets `finished = true`.
- **Consumer:** waits until "there is a box **or** the producer is finished". If the queue is empty at that point, everything is done and it leaves the loop. Otherwise it takes a box, **unlocks**, and then works on it.
- The `finished` flag is important. Without it the consumer would wait forever for a box that never comes.

TRY IT

Add a second consumer thread. Does every box still get consumed exactly once? (You will need `notify_all()` after setting `finished`, so **both** consumers wake up and stop.)

8. Futures and Promises: Getting a Result Back

A normal `std::thread` can't return a value. **Futures** solve that.

> **KEY IDEA**
>
> **A cloakroom ticket.** You hand in your coat and get a ticket (the **future**). Later you show the ticket and get your coat (the **result**). If it isn't ready yet, you wait at the counter.

8.1 std::async — the easy way

08_async.cpp

```cpp
#include <future>
#include <iostream>
int slow_square(int x) {
    return x * x;               // imagine this takes a long time
}
int main() {
    // start slow_square(7) on another thread; get a "ticket" for the answer
    std::future<int> ticket = std::async(std::launch::async, slow_square, 7);
    std::cout << "Main is doing other work...\n";
    int answer = ticket.get();  // wait for the result and collect it
    std::cout << "Answer: " << answer << "\n";
}
```

- `std::async(std::launch::async, slow_square, 7)` runs `slow_square(7)` on another thread and gives you a `std::future<int>` straight away.
- `ticket.get()` waits until the answer is ready and returns it. You can call `get()` **only once**.
- Always write `std::launch::async`, otherwise C++ is allowed to delay the work until you call `get()`.
- Bonus: if the function throws an exception, `get()` throws it in `main`, so errors are not lost.

8.2 std::promise — sending the value yourself

A **promise** is the other end of a future: one thread **sets** the value, another thread **gets** it.

09_promise.cpp

```cpp
#include <future>
#include <iostream>
#include <thread>
void worker(std::promise<int> p) {
    int result = 6 * 7;
    p.set_value(result);        // keep the promise: send the value
}
int main() {
    std::promise<int> p;                    // the sending end
    std::future<int> f = p.get_future();    // the receiving end
    std::thread t(worker, std::move(p));    // give the promise to the thread
    std::cout << "Result: " << f.get() << "\n";  // waits for set_value
    t.join();
}
```

Piece	Role
`std::promise<int>`	The sending end. Call `set_value(...)` once.
`std::future<int>`	The receiving end. Call `get()` to wait for the value.
`std::move(p)`	A promise can’t be copied, only moved into the thread.

Use `async` when you just want to run a function and get its result. Use a `promise` when the value is produced somewhere in the middle of a thread’s work.

9. Atomics

For a **single simple variable** like a counter or a true/false flag, a mutex is more than you need. `std::atomic` makes operations on that one variable **unbreakable**: `counter++` happens as one step that no other thread can interrupt.

10_atomic.cpp

```cpp
#include <atomic>
#include <iostream>
#include <thread>
std::atomic<int> counter = 0;   // atomic: ++ is one unbreakable step
void add_many() {
    for (int i = 0; i < 100000; i++) {
        counter++;              // safe without a mutex
    }
}
int main() {
    std::thread a(add_many);
    std::thread b(add_many);
    a.join();
    b.join();
    std::cout << "counter = " << counter << "\n";  // always 200000
}
```

Same program as Section 3, but `int` became `std::atomic<int>`. No mutex, and the answer is always 200,000.

Use	When
`std::atomic`	One simple variable: a counter, a flag like `std::atomic<bool> stop`.
`std::mutex`	Several variables that must change together, or any container (`vector`, `queue`, `map`).

PITFALL

Atomics only protect **one operation on one variable**. `if (x == 0) x = 1;` is two operations — another thread can change `x` in between. When in doubt, use a mutex.

10. Thread-Safe Classes

A class is **thread-safe** if many threads can use the same object at once without bugs. The simplest way: put the mutex **inside** the class, and lock it in every method.

11_safe_counter.cpp

```
#include <iostream>
#include <mutex>
#include <thread>
class SafeCounter {
public:
    void add(int n) {
        std::lock_guard<std::mutex> lock(m_);
        value_ += n;
    }
    int get() {
        std::lock_guard<std::mutex> lock(m_);
        return value_;          // return a copy, not a reference
    }
private:
    std::mutex m_;              // the lock lives with the data
    int value_ = 0;
};
int main() {
    SafeCounter c;
    std::thread a([&c] { for (int i = 0; i < 1000; i++) c.add(1); });
    std::thread b([&c] { for (int i = 0; i < 1000; i++) c.add(1); });
    a.join();
    b.join();
    std::cout << "total = " << c.get() << "\n";   // 2000
}
```

Three simple rules for thread-safe classes:

1. **Keep the mutex private, next to the data.** Users of the class should never have to lock anything themselves.
2. **Return copies, not references.** A reference to the inside data would let someone use it after the lock is gone.
3. **Do the whole job in one method.** Checking and then acting in two separate calls is not safe:

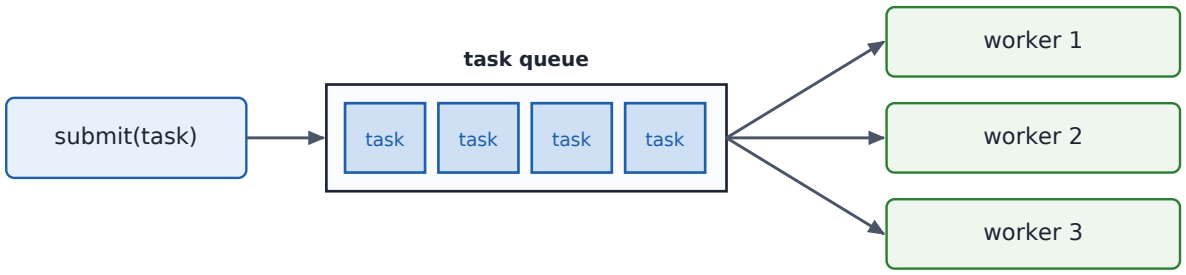
```
if (!q.empty()) {     // another thread may take the last item here...
    x = q.pop();      // ...so this can fail
}
// Better: one method that checks AND takes, while holding the lock:
bool try_pop(int& x);
```

11. Project: A Simple Thread Pool

11.1 What is a thread pool?

Starting a new thread for every small job is slow. A **thread pool** starts a few **worker threads once**. You give jobs (tasks) to the pool; they go into a **queue**; free workers take them out and run them.

> **KEY IDEA**
>
> **A team of waiters in a restaurant.** The restaurant doesn't hire a new waiter for every customer. It has three waiters. Orders go on a board (the queue). Whenever a waiter is free, they take the next order.



Tasks go into the queue; worker threads take them out and run them.

Notice: this is just the **producer-consumer** pattern from Section 7. The code that calls `submit` is the producer; the workers are the consumers; the tasks are the boxes.

11.2 Build it in 4 steps

Try to write each step yourself before looking at the full code in 11.3.

1. **The members.** A `vector<std::thread>` for workers, a `queue<std::function<void()>>` for tasks (a `std::function<void()>` can hold any function that takes nothing and returns nothing, including lambdas), a `mutex`, a `condition_variable`, and a `bool stopping` flag.
2. **The constructor.** Start N threads. Each one runs a function `worker_loop()`.
3. `worker_loop()`**.** Forever: lock; wait until "stopping **or** there is a task"; if stopping and the queue is empty, return; otherwise take one task out; **unlock**; run the task. (This is the consumer from Section 7.)
4. `submit(task)` **and the destructor.** `submit`: lock, push, unlock, `notify_one()`. Destructor: lock, set `stopping = true`, unlock, `notify_all()`, then `join()` every worker.

11.3 The complete thread pool

12_thread_pool.cpp

```cpp
#include <condition_variable>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
class ThreadPool {
public:
    ThreadPool(int num_threads) {
        for (int i = 0; i < num_threads; i++) {
            workers_.push_back(std::thread([this] { worker_loop(); }));
        }
    }
    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(m_);
            stopping_ = true;               // 1. tell workers to stop
        }
        cv_.notify_all();                   // 2. wake every worker
        for (auto& w : workers_) {
            w.join();                       // 3. wait for them to finish
        }
    }
    void submit(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(m_);
            tasks_.push(task);              // put the task in the queue
        }
        cv_.notify_one();                   // wake one sleeping worker
    }
private:
    void worker_loop() {
        while (true) {
            std::function<void()> task;
            {
                std::unique_lock<std::mutex> lock(m_);
                cv_.wait(lock, [this] { return stopping_ || !tasks_.empty(); });
                if (stopping_ && tasks_.empty()) return;  // nothing left
                task = tasks_.front();
                tasks_.pop();
            }                               // unlock BEFORE running
            task();                         // run the job
        }
    }
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex m_;
    std::condition_variable cv_;
    bool stopping_ = false;
};
int main() {
    ThreadPool pool(3);
    for (int i = 1; i <= 6; i++) {
        pool.submit([i] {
            std::cout << "Task " + std::to_string(i) + " done\n";
        });
    }
}   // pool is destroyed here: it finishes all tasks, then stops
```

11.4 The important details

- **Run the task after unlocking.** If a worker held the lock while running a task, no other worker could take a task, and the pool would do only one thing at a time.
- **The destructor finishes all tasks first.** Workers only quit when `stopping` is true **and** the queue is empty.
- `notify_all()` **in the destructor**, because **every** sleeping worker must wake up to see `stopping` and quit.
- `[this]` in the lambda lets the new thread call the member function `worker_loop()` of this pool.

11.5 Make it your own

1. Type the pool from memory using the 4 steps. Compare with 11.3.
2. Submit 20 tasks that each print their number and the worker's id (`std::this_thread::get_id()`).
3. Use the pool to add up a big `vector` in 4 pieces. Each task adds its piece into a `std::atomic<long>`.
4. **Bonus (harder):** make `submit` return a `std::future` so callers can get a result back. Hint: wrap the task in a `std::packaged_task`, which works like a promise that is filled in by running a function.

12. Summary

12.1 Which tool do I use?

I want to…	Use
Run code at the same time	`std::thread` + `join()`
Protect shared data	`std::mutex` + `std::lock_guard`
Lock two mutexes	`std::scoped_lock`
Unlock early, or use a condition variable	`std::unique_lock`
Wait until something happens	`std::condition_variable` + `wait(lock, condition)`
Pass work between threads	a queue + mutex + condition variable (producer–consumer)
Get a result back from a thread	`std::async` + `std::future` (or `std::promise`)
Share one simple counter or flag	`std::atomic`
Run many small tasks efficiently	a thread pool

12.2 The most common mistakes

Mistake	What happens	Fix
Forgetting `join()`	Program crashes	Join every thread you start
Changing shared data without a lock	Wrong results (race condition)	Use a mutex or atomic
Calling `lock()`/`unlock()` by hand	Program freezes if unlock is skipped	`lock_guard`
Locking two mutexes in different orders	Deadlock (freeze)	`scoped_lock` or same order everywhere
`wait` without a condition	Wakes up when it shouldn't	`cv.wait(lock, [] { return ...; })`
Doing slow work while holding a lock	Threads wait, program gets slow	Unlock first, then work
No "finished/stop" flag	Consumer or worker waits forever	Add a flag + `notify_all()`

12.3 Finding race conditions automatically

Compile with `-fsanitize=thread` (works with g++ and clang++). If two threads touch the same data unsafely, the program prints a warning showing the exact lines — even when the answer happened to look right. Try it on `03_race.cpp`:

```
g++ -std=c++17 -g -pthread -fsanitize=thread 03_race.cpp -o race
./race
```

13. Where to Learn More

Use these only if you want more explanation of a topic. Start with the first two.

Resource	Why
“Back to Basics: Concurrency” — Arthur O’Dwyer, CppCon 2020 (free on YouTube)	One hour, clear, covers almost everything in this guide.
cppreference.com — “Concurrency support library”	Look up any function you are unsure about. Each page has a small example.
OSTEP (free book), chapters 26–30 — pages.cs.wisc.edu/~remzi/OSTEP/	Threads, locks and condition variables explained simply. Also useful for your OS course later.
Anthony Williams, *C++ Concurrency in Action*, 2nd edition, chapters 1–4	The standard book on this topic, for when you want to go deeper.

RULE OF THUMB

Best way to learn this week: read a section, close the guide, type the example from memory, run it several times, then change something and predict what will happen.